Information security – tutorial 3

1. Calculating Hashes

I created three files:

- an empty file
- a file containing “a”
- a file containing “abc”

Then I calculated hashes using MD5, SHA-256, and SHA3-256.

```
hiba@hiba-VMware20-1:~/tutorial3$ openssl md5 empty a.txt abc.
txt
MD5(empty)= d41d8cd98f00b204e9800998ecf8427e
MD5(a.txt)= 0cc175b9c0f1b6a831c399e269772661
MD5(abc.txt)= 900150983cd24fb0d6963f7d28e17f72
```

```
hiba@hiba-VMware20-1:~/tutorial3$ openssl sha256 empty a.txt a
bc.txt
SHA2-256(empty)= e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b9
34ca495991b7852b855
SHA2-256(a.txt)= ca978112ca1bbdcafac231b39a23dc4da786eff8147c4
e72b9807785afee48bb
SHA2-256(abc.txt)= ba7816bf8f01cfea414140de5dae2223b00361a3961
77a9cb410ff61f20015ad
```

```
hiba@hiba-VMware20-1:~/tutorial3$ openssl sha3-256 empty a.txt
 abc.txt
SHA3-256(empty)= a7ffc6f8bf1ed76651c14756a061d662f580ff4de43b4
9fa82d80a4b80f8434a
SHA3-256(a.txt)= 80084bf2fba02475726feb2cab2d8215eab14bc6bdd8b
fb2c8151257032ecd8b
SHA3-256(abc.txt)= 3a985da74fe225b2045c172d6bd390bd855f086e3e9
d525b46bfe24511431532
```

The results show that:

- Each input produces a unique hash
- The same input always produces the same output

This shows that hash functions are deterministic.

2. Avalanche Effect

I created two files:

- b.txt containing “B”
- c.txt containing “C”

```
hiba@hiba-VMware20-1:~/tutorial3$ echo -n "B" > b.txt
hiba@hiba-VMware20-1:~/tutorial3$ cat b.txt
Bhiba@hiba-VMware20-1:~/tutorial3$ echo -n "C" > c.txt
hiba@hiba-VMware20-1:~/tutorial3$ cat c.txt
Chiba@hiba-VMware20-1:~/tutorial3$ xxd -b b.txt
00000000: 01000010
   B
hiba@hiba-VMware20-1:~/tutorial3$ xxd -b c.txt
00000000: 01000011
   C
```

Binary values:

- B = 01000010
- C = 01000011

They differ by only **one bit**.

Then I calculated SHA-256:

```
hiba@hiba-VMware20-1:~/tutorial3$ openssl sha256 b.txt c.txt
SHA2-256(b.txt)= df7e70e5021544f4834bbee64a9e3789febc4be81470d
f629cad6ddb03320a5c
SHA2-256(c.txt)= 6b23c0d5f35d1b11f9b683f0b0a617355deb11277d91a
e091d399c655b87940d
```

Even though the input difference is very small, the hash outputs are completely different.

This demonstrates the **avalanche effect**:

A small change in input leads to a large change in output.

3. MD5 Collisions

I extracted the files:

- msg1
- msg2

Then I compared them:

```
hiba@hiba-VMware20-1:~/Downloads$ cmp msg1 msg2
msg1 msg2 differ: byte 2, line 1
```

The result shows that the files are different.

Next, I calculated MD5:

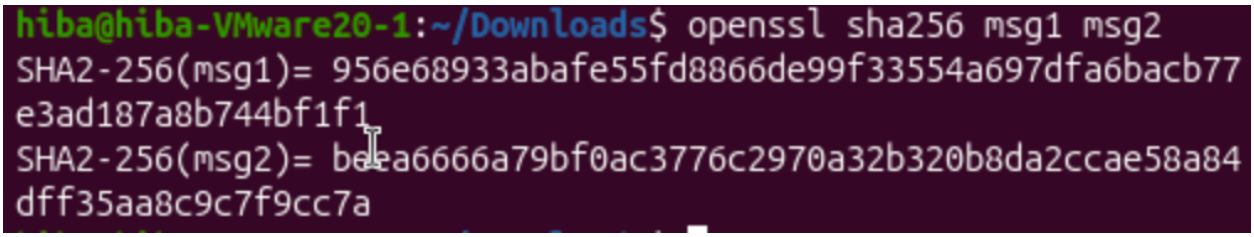
```
hiba@hiba-VMware20-1:~/Downloads$ openssl md5 msg1 msg2
MD5(msg1)= abaf0d74788fd8f969bf992c7d0456ad
MD5(msg2)= abaf0d74788fd8f969bf992c7d0456ad
```

Both files produced the **same MD5 hash**.

This is called a **collision**:

Two different files produce the same hash.

Then I tested SHA-256:



The hashes are different.

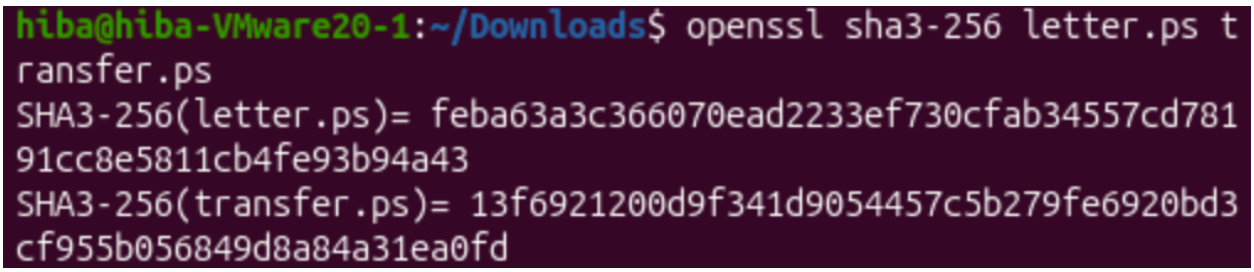
This shows:

- MD5 is broken and insecure
- SHA-256 is more secure

4. **MD5 Collision Modification**

I checked the files letter.ps and transfer.ps using MD5.

Both files produced the same MD5 hash, even though they are different files.



I also calculated the SHA3-256 hash of both files.

The results are different, even though the MD5 hashes are the same.

This shows that the collision only applies to MD5, and stronger hash functions like SHA3-256 do not have this problem.

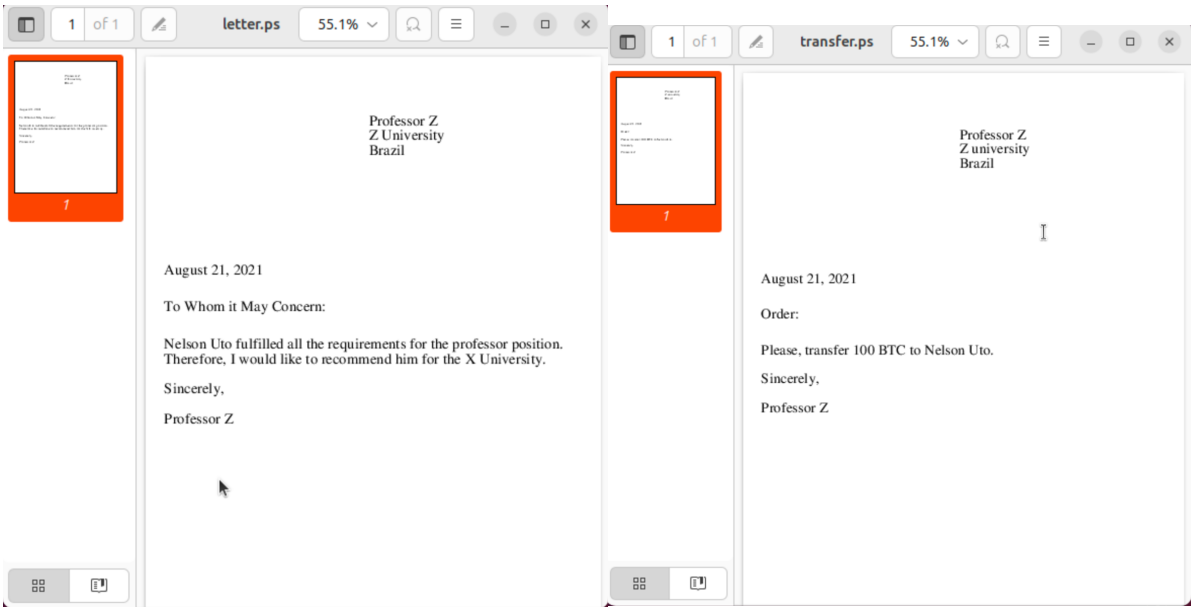
This demonstrates a collision in MD5.
Even though the files are not identical, they still have the same hash value.

This shows that MD5 is not secure and should not be used for integrity checking.

```
hiba@hiba-VMware20-1:~/Downloads$ openssl md5 letter.ps transf
er.ps
MD5(letter.ps)= fe2e5cd04e1d3da4851794d2384b2a80
MD5(transfer.ps)= fe2e5cd04e1d3da4851794d2384b2a80
hiba@hiba-VMware20-1:~/Downloads$ cmp letter.ps transfer.ps
letter.ps transfer.ps differ: byte 84, line 3
```

Creating or modifyinf such files requires techniques, wich shows how attackers can exploit MD5.

I opened the files letter.ps and transfer.ps and confirmed that their content are different. One file contains a recommendation letter, while the other contains a payment request.



Despite this, both have the same MD5 hash, wish demonstrates a collision. This shows that MD5 is not secure and can be exploited.

4. Hash number

1 – Malware

2 - Malware

3 - Malware

4 -Malware

5 -Malware

6 – Malware

7 -Malware

8 -Malware

9 -Malware

10 -Malware

I searched the given SHA-256 hashes using VirusTotal. All of them were identified as malware. This shows how hash can be used to identify malicious files without opening them.

5. Conclusion

In this lab, I learned that:

- Hash functions create a unique fingerprint of data
- Small changes in input create large differences in output (avalanche effect)
- MD5 is not secure because collisions can happen
- SHA-256 is more secure and should be used instead